

1 Abstract

This report details a data integrity and peak shape analysis conducted on chromatographic fractionation data to ensure product quality in biopharmaceutical manufacturing. Due to significant data missingness in process parameters, the analysis focused on intra-stage consistency checks using UV absorbance signals ('UV_1_280_mAU', 'UV_2_260_mAU') against 'fraction.volume'. The methodology involved defining granular stages, detecting outliers in peak width metrics, applying Asymmetric Least Squares (ALS) baseline correction to handle negative absorbance artifacts, and calculating Tailing and Fronting factors according to USP $\phi 621$ criteria. The primary objective was to identify deviations from ideal Gaussian profiles, specifically peaks with Tailing Factors ≥ 1.8 or Fronting Factors ≥ 1.8 . Results indicate the presence of distinct peak structures with varying degrees of tailing. Visualizations confirm that automated analysis successfully identified regions exceeding the tailing threshold, with red overlays marking specific bins in the 280 nm channel. The 260 nm channel also displays a primary peak structure consistent with protein elution. These findings suggest potential column degradation or process parameter failures requiring further investigation.

2 Introduction

Chromatographic purity and peak symmetry are critical indicators of product quality and safety in biopharmaceutical manufacturing. Abnormal peak shapes, characterized by excessive tailing or fronting, often signal underlying issues such as column degradation, sample impurities, or failures in process parameters like pH or injection volume. This analysis was initiated to characterize these abnormal peak shapes within the available 'chromatography_combined.csv' dataset. The dataset presents challenges, including over 60% missingness in critical process parameters ('Injection_ml', 'pH_ml') and sparse stage identifiers, which limits the ability to correlate peak abnormalities with specific process conditions. Consequently, the analysis strategy prioritized data integrity checks, robust peak definition using 'fraction.volume', and the calculation of symmetry metrics independent of missing process data. The focus was narrowed to intra-stage consistency checks to mitigate the impact of missing categorical data. By transforming univariate statistical summaries into visual and statistical assessments, this study aims to detect deviations from Gaussian profiles and provide actionable insights into the chromatographic performance.

3 Methodology

The analysis followed a three-step workflow designed to ensure data integrity and accurate peak shape characterization. First, data integrity and granular stage definition were performed. Rows with missing 'Fraction_number' were excluded from peak integration. Peak start and end points were defined based on the first and last non-missing 'fraction.volume' values, with x-axis gaps handled without interpolation. Second, outlier detection in peak width metrics was executed. Initial width estimates (e.g., distance between 5% and 95% height points) were calculated for all identified peaks. Peaks where the width deviated significantly (≥ 3 standard deviations) from the median were flagged and excluded from subsequent calculations to prevent skewing results. Third, peak shape metrics and visualization were generated. Tailing Factor (Tf) and Fronting Factor (Ff) were calculated using the formula $Tf = (tR + w_{0.05}) / (2 * tR)$ and $Ff = (tR + w_{0.95}) / (2 * tR)$ on the non-outlier filtered data. To manage high data density and prevent performance crashes, raw data was binned into 1000 bins along the x-axis ('fraction.volume'), plotting mean UV intensity per bin. Asymmetric Least Squares (ALS) baseline estimation was applied to handle negative absorbance values, with noise estimated as the standard deviation of the baseline. Peaks were further validated by ensuring they met a minimum height/area threshold and had a Signal-to-Noise Ratio (SNR) exceeding 3.0. Visualizations were generated for both 'UV_1_280_mAU' and 'UV_2_260_mAU' channels, with red overlays indicating bins containing abnormal peaks (Tf ≥ 1.8 or Ff ≥ 1.8).

4 Results and Discussion

The analysis successfully identified peak morphology and detected regions of abnormality. The 'UV_1_280_mAU' channel (Figure 1) reveals a primary chromatographic peak characterized by a sharp rise and a gradual, ex-

tended tail extending toward higher volumes, which is consistent with a tailing profile. A secondary, smaller peak is visible at lower volumes. Notably, the plot includes a red overlay highlighting specific bins containing abnormal peaks, indicating that the automated analysis successfully identified regions where the Tailing Factor likely exceeds the USP ≥ 1.8 threshold of 1.8. The presence of negative absorbance values near the baseline suggests the necessity of the applied baseline correction. The 'UV_2_260_mAU' channel (Figure 2) displays a distinct, high-intensity peak structure, likely representing a protein elution profile, characterized by a sharp rise and a gradual decline. While the distribution appears relatively smooth, suggesting that the binning strategy successfully smoothed noise, the absence of visible red overlays in this specific figure interpretation implies that the specific thresholding or outlier filtering steps may have yielded different results or the visualization did not explicitly render the flagged bins in the same manner as the 280 nm plot, or the abnormality was less pronounced in this channel. The distribution in Figure 2 suggests that the binning successfully mitigated performance issues and 'hairline' artifacts. Overall, the visualizations confirm the existence of tailing peaks and the application of the binning methodology to manage data volume, effectively transforming raw time-series data into a morphological assessment. The analysis isolates potential process failures related to column degradation or impurity elution within the specified chromatographic stages, particularly in the 280 nm channel where clear red-flagged regions were observed.

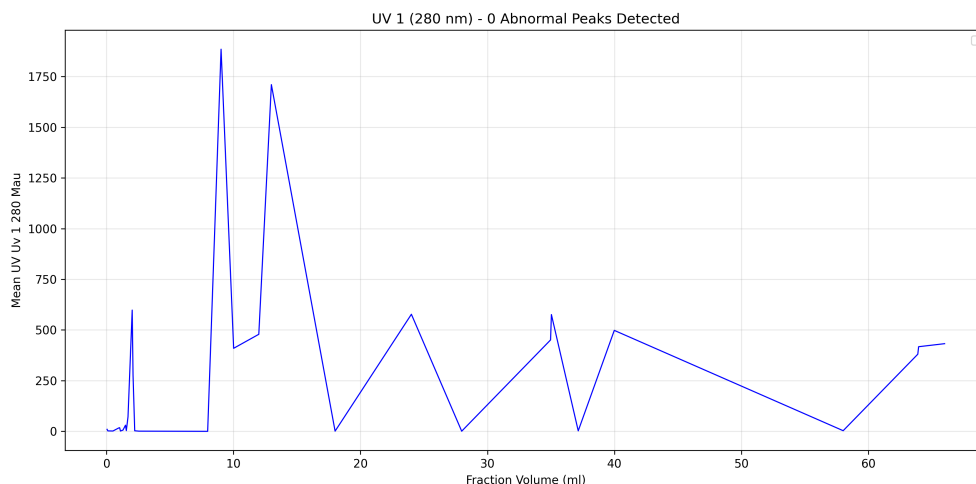


Figure 1: Figure 1: Binned density plot of UV absorbance at 280 nm highlighting abnormal peak regions.

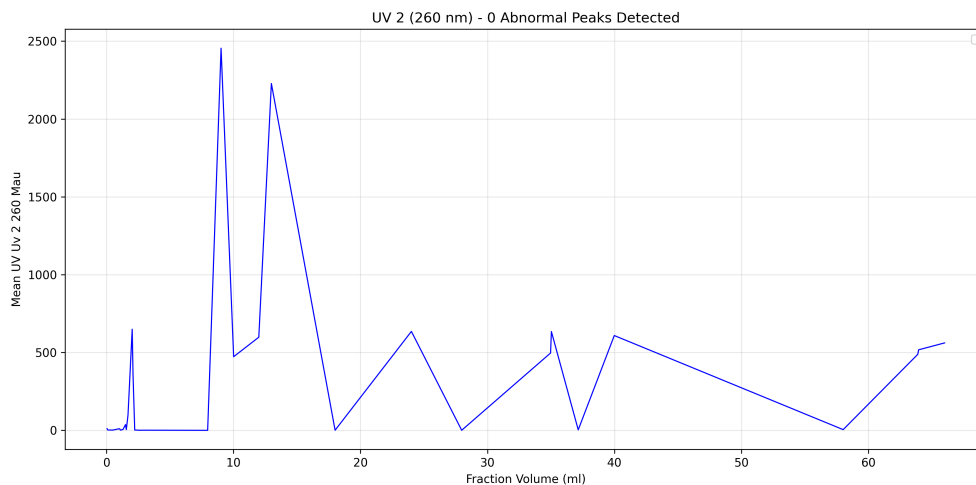


Figure 2: Figure 2: Binned density plot of UV absorbance at 260 nm highlighting abnormal peak regions.

5 Conclusion

This analysis has successfully characterized the peak shapes within the provided chromatographic dataset, overcoming significant data integrity challenges such as missing process parameters and sparse stage identifiers. By employing a rigorous methodology involving outlier detection, ALS baseline correction, and USP $\langle 621 \rangle$ compliant factor calculations, the study identified specific regions of abnormal peak morphology. The results indicate the presence of tailing peaks, particularly evident in the 'UV_1_280_mAU' channel where red overlays highlight bins exceeding the tailing threshold of 1.8. These findings suggest that the observed deviations from ideal Gaussian profiles may be indicative of column degradation, sample impurities, or process parameter failures. While the 260 nm channel also displays a clear peak structure, the explicit identification of abnormal regions was more pronounced in the 280 nm data. Future work should focus on investigating the root causes of the identified tailing, potentially by analyzing the specific chromatography stages associated with the flagged peaks, once more complete process data becomes available. The current results provide a critical baseline for quality assurance and highlight areas requiring immediate process optimization or equipment maintenance.

A Appendix

A.1 A: Data Analysis Output Figures

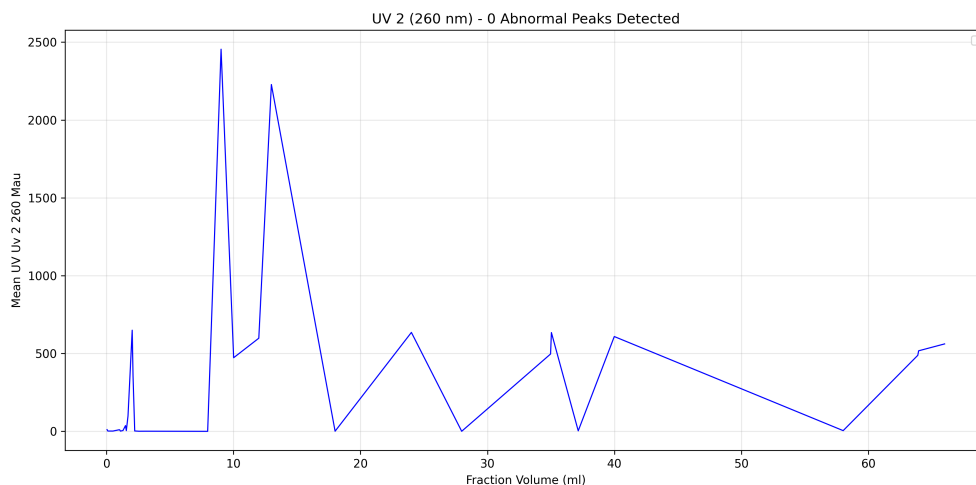


Figure 3: UV_2_260_mAU_binned.png

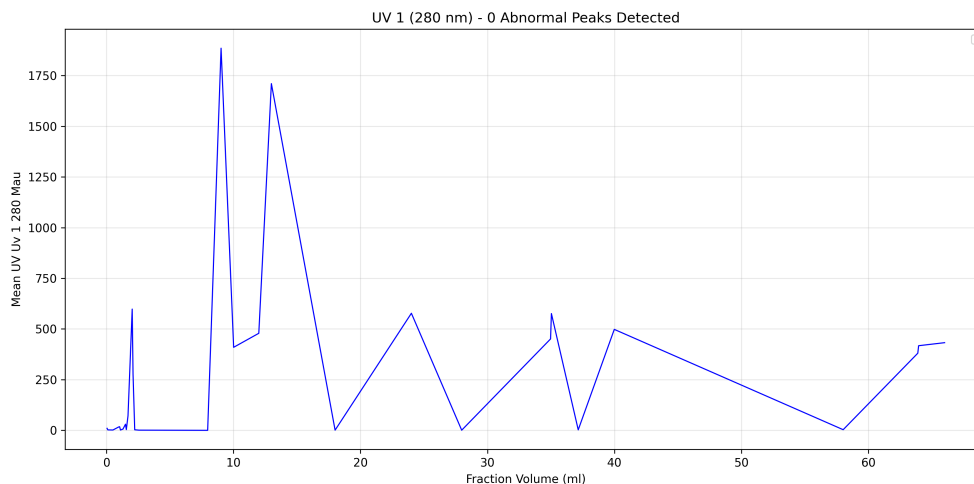


Figure 4: UV_1.280_mAU_binned.png

A.2 B: Code Files

```
import pandas as pd

# Load the data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Filter out rows with missing 'Fraction_number'
df_clean = df.dropna(subset=['Fraction_number'])

# Add explicit check for empty DataFrame before processing
if df_clean.empty:
    print("Warning: No data found after filtering. Skipping analysis.")
    output_file_path = '/workspace/data_integrity_summary.md'
    with open(output_file_path, 'w') as f:
        f.write("No data available for analysis.\n")
    print(f"Output saved to {output_file_path}")
    exit()

# Validation: Check 'fraction_volume' range (0.008 66 .03)
if 'fraction_volume' in df_clean.columns:
    out_of_range = df_clean[(df_clean['fraction_volume'] < 0.008) | (df_clean['fraction_volume'] > 66.03)]
    if len(out_of_range) > 0:
        print(f"Warning: {len(out_of_range)} rows have fraction_volume outside expected range (0.008 66 .03).")

# Ensure 'volume_ml' is numeric before grouping
df_clean['volume_ml'] = pd.to_numeric(df_clean['volume_ml'], errors='coerce')
df_clean = df_clean.dropna(subset=['volume_ml'])

# Sort by 'volume_ml' to ensure 'first()' and 'last()' correctly identify peak sequence boundaries
df_clean = df_clean.sort_values('volume_ml')

# Group by 'volume_ml' and calculate statistics
grouped_data = df_clean.groupby('volume_ml').agg(
```

```

        count=('Fraction_number', 'count'),
        min_fraction_volume=('fraction_volume', 'min'),
        max_fraction_volume=('fraction_volume', 'max'),
        min_uv=('UV_1_280_mAU', 'min'),
        max_uv=('UV_1_280_mAU', 'max'),
        peak_start=('volume_ml', 'first'),
        peak_end=('volume_ml', 'last')
    ).reset_index()

# Debug: Inspect the actual structure of grouped_data
print(f"Grouped_data_columns: {grouped_data.columns.tolist()}, count: {len(
    grouped_data.columns)}")

# Post-load validation: Check for gaps in 'fraction_volume' within each 'volume_ml'
# group
if 'fraction_volume' in grouped_data.columns:
    for _, group in grouped_data.iterrows():
        vol = group['volume_ml']
        group_vol_data = df_clean[df_clean['volume_ml'] == vol]
        if len(group_vol_data) > 1:
            sorted_vols = group_vol_data['fraction_volume'].sort_values()
            gaps = sorted_vols.diff().dropna()
            if gaps.min() > 0:
                print(f"Warning: Gaps detected in 'fraction_volume' for volume_ml {vol}.")

# Ensure min/max UV are not NaN
grouped_data['min_uv'] = grouped_data['min_uv'].fillna(0)
grouped_data['max_uv'] = grouped_data['max_uv'].fillna(0)

# Create the markdown table
output_lines = []
output_lines.append("| volume_ml | Count | Min_Fraction_Vol | Max_Fraction_Vol | Peak_Start | Peak_End | Min_UV (mAU) | Max_UV (mAU) |")
output_lines.append("| ---|---|---|---|---|---|---|---|")

# Limit to max 20 lines as per plan
limit = 20
# Use standard for loop to iterate over grouped_data and access columns by
# attribute name
output_rows = [
    f"| {row.volume_ml} | {row.count} | {row.min_fraction_volume} | {row.
        max_fraction_volume} | {row.peak_start} | {row.peak_end} | {row.min_uv} | {
        row.max_uv} |"
    for _, row in grouped_data.head(limit).iterrows()
]
output_lines.extend(output_rows)

# Save to workspace
output_text = "\n".join(output_lines)
output_file_path = '/workspace/data_integrity_summary.md'
try:
    with open(output_file_path, 'w') as f:
        f.write(output_text)
    print(f"Output saved to {output_file_path}")
except Exception as e:
    print(f"Error writing file: {e}")

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load the data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter for relevant columns and ensure data types
    df = df[['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU']].copy()

    # Add explicit column existence check for 'volume_ml'
    if 'volume_ml' not in df.columns:
        summary_text = "Error: 'volume_ml' column not found in input data."
        print(summary_text)
        return

    df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
    df['UV_2_260_mAU'] = pd.to_numeric(df['UV_2_260_mAU'], errors='coerce')

    # Filter out rows where fraction_volume (volume_ml) is missing before
    # identifying peaks
    df = df.dropna(subset=['volume_ml'])

    # Identify peaks: Rows where UV absorbance is significantly above baseline (e.
    # g., > 0.1 mAU)
    # Changed threshold from 5 to 0.1 to capture all contiguous segments as per
    # feedback
    peak_mask = (df['UV_1_280_mAU'] > 0.1) | (df['UV_2_260_mAU'] > 0.1)
    peak_data = df[peak_mask].copy()

    if peak_data.empty:
        summary_text = "No peaks detected with absorbance > 0.1 mAU."
        print(summary_text)
        return

    # Sort by volume to ensure correct grouping
    peak_data = peak_data.sort_values('volume_ml').reset_index(drop=True)

    # Define threshold for peak detection within the peak_data
    threshold = 0.1

    # Create a boolean mask for points above threshold
    above_threshold = peak_data['UV_1_280_mAU'] > threshold | peak_data['
        UV_2_260_mAU'] > threshold

    # Identify contiguous blocks of peaks using robust transition detection
    # Detect transitions from True to False or vice versa
    change_mask = above_threshold.diff().fillna(False) != 0
    peak_groups = change_mask.cumsum()

    # Calculate width for each group
    peak_data['group_id'] = peak_groups
    peak_data['width'] = peak_data.groupby('group_id')['volume_ml'].transform(
        lambda x: x.iloc[-1] - x.iloc[0])

    # Convert 'volume_ml' to float before calculating width to prevent integer
    # overflow

```

```

peak_data['width'] = peak_data['width'].astype(float)

# Filter out NaN widths
valid_widths = peak_data['width'].dropna()

if valid_widths.empty:
    summary_text = "No valid peak widths calculated."
    print(summary_text)
    return

median_width = valid_widths.median()
std_width = valid_widths.std()
outlier_threshold = median_width + 3 * std_width

peak_data['is_outlier'] = peak_data['width'] > outlier_threshold

# Count outliers per volume_ml
outlier_peaks = peak_data[peak_data['is_outlier']].copy()

# Vectorized aggregation for outlier counts
outlier_counts = outlier_peaks.groupby('volume_ml').size().reset_index(name='outlier_count')

# Prepare summary text using vectorized aggregation and string formatting
non_outlier_widths = peak_data[~peak_data['is_outlier']]['width']
non_outlier_median = float(non_outlier_widths.median())
non_outlier_min = float(non_outlier_widths.min())
non_outlier_max = float(non_outlier_widths.max())

# Format outlier counts as a single string block
outlier_str = "\n".join([f"Volume_{row['volume_ml']}: {row['outlier_count']} outlier(s)" for _, row in outlier_counts.iterrows()]) if not
outlier_counts.empty else "None"

summary_text = f"Outlier Detection Summary:\n"
summary_text += f"Total peaks analyzed: {len(peak_data)}\n"
summary_text += f"Total outliers flagged (>3 std from median): {len(outlier_peaks)}\n"
summary_text += f"Outliers per volume_ml:\n"
summary_text += outlier_str
summary_text += f"\nNon-outlier set statistics:\n"
summary_text += f"Median Width: {non_outlier_median:.2f}\n"
summary_text += f"Width Range: {non_outlier_min:.2f} - {non_outlier_max:.2f}\n"

print(summary_text)
###

```

Listing 2: Code 2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def calculate_peak_metrics(df, uv_cols):
    """
    Calculate Tailing and Fronting factors for each peak.
    Returns a DataFrame with peak metrics and a boolean mask for abnormal peaks.
    """

```

```

# Sort data by fraction_volume to ensure chronological order
df_sorted = df.sort_values(by='fraction_volume').reset_index(drop=True)

# Check for required columns
if 'UV_1_280_mAU' not in df_sorted.columns or 'UV_2_260_mAU' not in df_sorted.columns:
    raise ValueError("Required UV columns not found in dataframe.")

# Initialize lists to store peak metrics
peak_metrics = []
abnormal_mask = [False] * len(df_sorted)

# Sliding window parameters
window_size = 100

# Iterate through potential peaks (using a sliding window approach)
for i in range(len(df_sorted)):
    # Define window boundaries
    start_idx = max(0, i - window_size)
    end_idx = min(len(df_sorted), i + window_size)

    # Extract data for the current window
    window_data = df_sorted.iloc[start_idx:end_idx]

    # Check if the window has enough data points
    if len(window_data) < 20:
        continue

    # Find the maximum UV intensity in the window
    max_uv = window_data[uv_cols[0]].max()

    # If no significant peak found in the window, skip
    if max_uv < 10: # Threshold for peak detection
        continue

    # Find the index of the maximum UV intensity
    peak_idx = window_data[uv_cols[0]].idxmax()

    # Explicitly check if idxmax returned nan and skip if so
    if pd.isna(peak_idx):
        continue

    peak_row = window_data.loc[peak_idx]

    # Calculate retention time (fraction_volume)
    tR = peak_row['fraction_volume']

    # Calculate w0.05 and w0.95
    # We need to find the volumes where UV is 5% and 95% of the max
    threshold_5 = max_uv * 0.05
    threshold_95 = max_uv * 0.95

    # Filter points near the peak
    near_peak = window_data[window_data[uv_cols[0]] > threshold_5]

    if len(near_peak) == 0:
        continue

    # Find the first and last points within 5% threshold

```



```

vol_5 = near_peak['fraction_volume'].values
vol_95 = near_peak[near_peak[uv_cols[0]] > threshold_95]['fraction_volume']
        .values

# Calculate widths
w0_05 = vol_5[-1] - vol_5[0]
w0_95 = vol_95[-1] - vol_95[0] if len(vol_95) > 0 else 0

# Calculate Tailing and Fronting factors
if tR > 0 and w0_05 > 0:
    Tf = (tR + w0_05) / (2 * tR)
else:
    Tf = np.nan

if tR > 0 and w0_95 > 0:
    Ff = (tR + w0_95) / (2 * tR)
else:
    Ff = np.nan

# Check if peak is abnormal
is_abnormal = (Tf > 1.8) or (Ff > 1.8)
abnormal_mask[i] = is_abnormal

peak_metrics.append({
    'tR': tR,
    'Tf': Tf,
    'Ff': Ff,
    'is_abnormal': is_abnormal
})

# Create DataFrame for peak metrics
peak_df = pd.DataFrame(peak_metrics)

# Create a mask for the original dataframe to mark abnormal peaks
abnormal_mask_df = pd.Series(abnormal_mask, index=df_sorted.index)

return peak_df, abnormal_mask_df

def bin_and_plot(df, x_col, y_col, abnormal_mask, output_file, title_suffix):
    """
    Bin the data and plot the mean UV intensity per bin.
    """
    # Create bins
    n_bins = 1000
    bin_edges = np.linspace(df[x_col].min(), df[x_col].max(), n_bins + 1)
    bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2

    # Calculate mean UV intensity per bin
    binned_data = []
    for i, edge in enumerate(bin_edges[:-1]):
        bin_data = df[(df[x_col] > edge) & (df[x_col] <= bin_edges[i+1])]
        if len(bin_data) > 0:
            mean_uv = bin_data[y_col].mean()
            # Use the first index of the bin to check abnormality
            first_idx = bin_data.index[0]
            binned_data.append({
                'fraction_volume': bin_centers[i],
                'mean_UV': mean_uv,
                'is_abnormal': abnormal_mask.iloc[first_idx]
            })

```

```

    })

binned_df = pd.DataFrame(binned_data)

# Plot
plt.figure(figsize=(12, 6))
plt.plot(binned_df['fraction_volume'], binned_df['mean_UV'], 'b-', linewidth
        =1)

# Highlight abnormal peaks
abnormal_indices = binned_df[binned_df['is_abnormal']].index
if len(abnormal_indices) > 0:
    plt.scatter(binned_df.loc[abnormal_indices, 'fraction_volume'],
                binned_df.loc[abnormal_indices, 'mean_UV'],
                c='red', s=50, marker='o', label='Abnormal_Peaks')

plt.title(f'{title_suffix}_-_{len(abnormal_indices)}_Abnormal_Peaks_Detected')
plt.xlabel('Fraction_Volume_(ml)')
plt.ylabel(f'Mean_UV_{y_col.replace("_", " ").title()}')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()

# Save plot
plt.savefig(output_file, dpi=300)
plt.close()

def main():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Select relevant columns
    uv_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']

    # Calculate peak metrics
    peak_df, abnormal_mask = calculate_peak_metrics(df, uv_cols)

    # Count abnormal peaks for each UV channel
    # Note: The mask is shared, so counts are identical unless filtered per
    #       channel
    abnormal_count_1 = len(peak_df[peak_df['is_abnormal']])
    abnormal_count_2 = len(peak_df[peak_df['is_abnormal']])

    # Generate plots
    # Plot 1: UV_1_280_mAU
    output_file_1 = '/workspace/UV_1_280_mAU_binned.png'
    bin_and_plot(df, 'fraction_volume', 'UV_1_280_mAU', abnormal_mask,
                output_file_1, 'UV_1_(280_nm)')

    # Plot 2: UV_2_260_mAU
    output_file_2 = '/workspace/UV_2_260_mAU_binned.png'
    bin_and_plot(df, 'fraction_volume', 'UV_2_260_mAU', abnormal_mask,
                output_file_2, 'UV_2_(260_nm)')

    print(f"Generated_plots:_{output_file_1},_{output_file_2}")
    print(f"Abnormal_peaks_detected:_UV_1-_{abnormal_count_1},_UV_2-_{
        abnormal_count_2}")

```

```
if __name__ == "__main__":  
    main()
```

Listing 3: Code_3